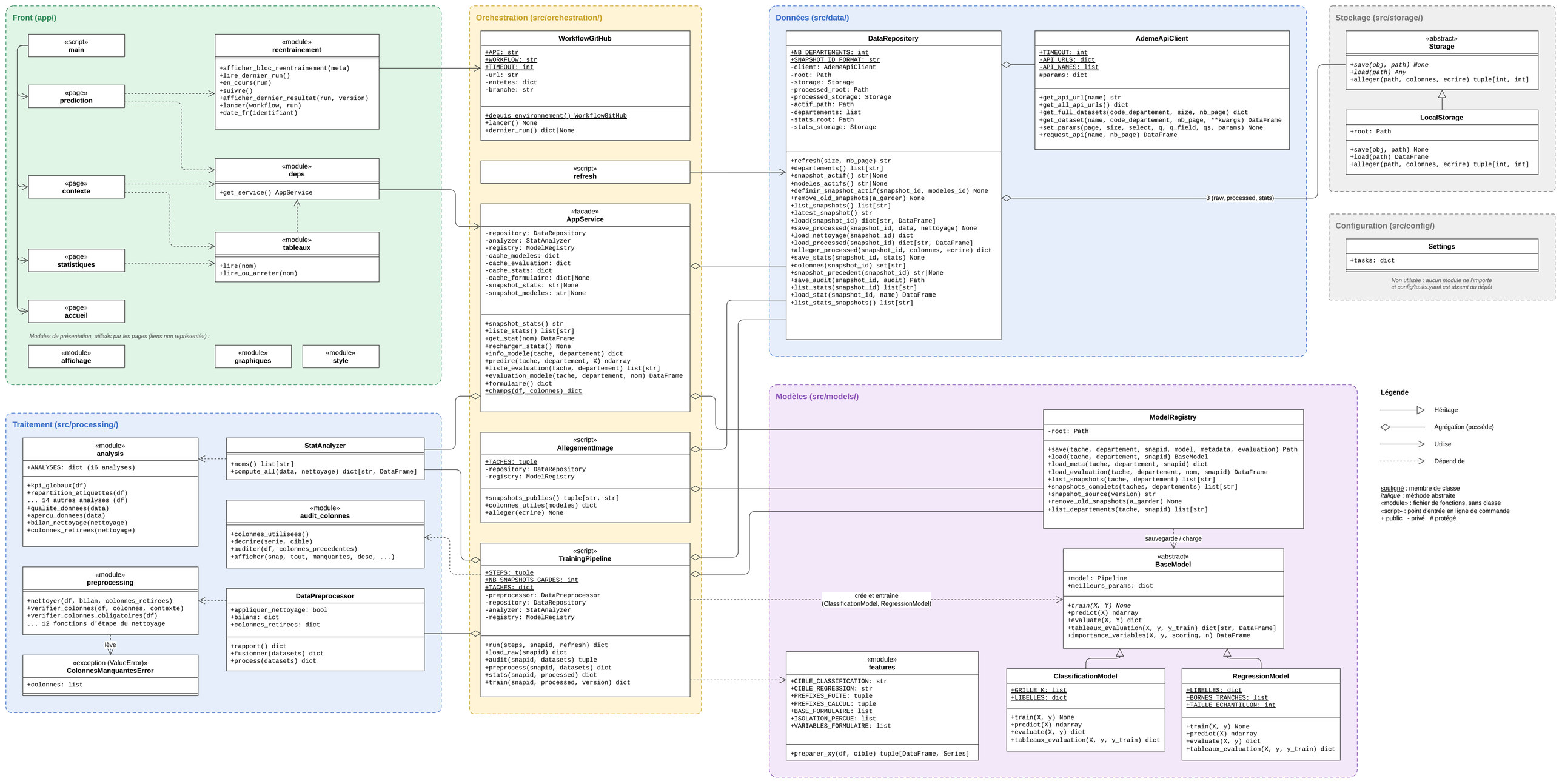
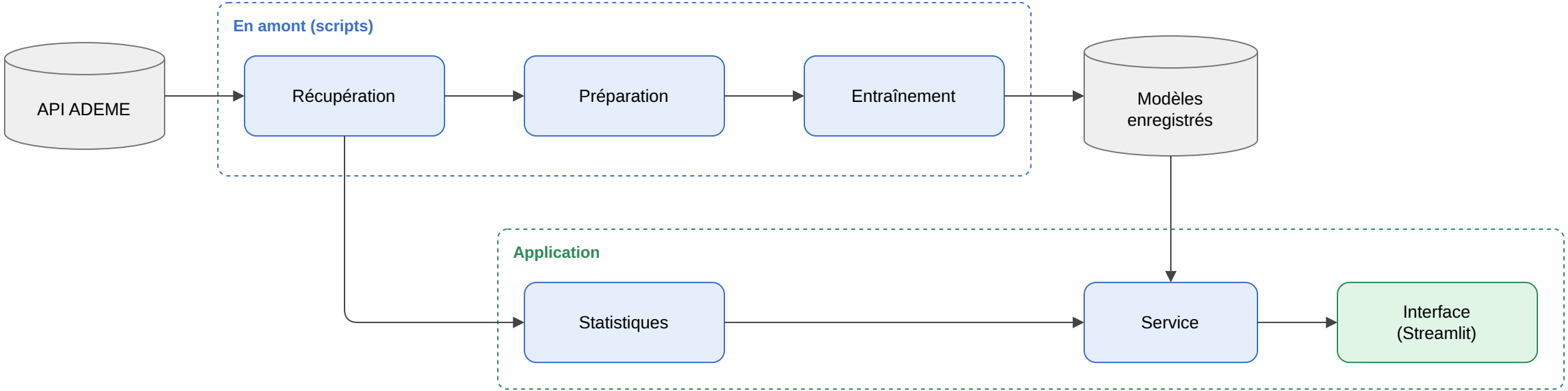

Front (app/)
«script» main
«module» reentrainement
+afficher_bloc_reentrainement(meta)
+lire_dernier_run()
+en_cours(run)
+suivre()
+afficher_dernier_resultat(run, version)
+lancer(workflow, run)
+date_fr(identifiant)
«page» prediction
«module» deps
+get_service() AppService
«page» contexte
«module» tableaux
+lire(nom)
+lire_ou_arreter(nom)
«page» statistiques
«page» accueil
Modules de présentation, utilisés par les pages (liens non représentés) :
«module» affichage
«module» graphiques
«module» style
Orchestration (src/orchestration/)
WorkflowGitHub
+API: str
+WORKFLOW: str
+TIMEOUT: int
-url: str
-entetes: dict
-branche: str
+depuis_environnement() WorkflowGitHub
+lancer() None
+dernier_run() dict|None
«script» refresh
«facade» AppService
-repository: DataRepository
-analyzer: StatAnalyzer
-registry: ModelRegistry
-cache_modeles: dict
-cache_evaluation: dict
-cache_stats: dict
-cache_formulaire: dict|None
-snapshot_stats: str|None
-snapshot_modeles: str|None
+snapshot_stats() str
+liste_stats() list[str]
+get_stat(nom) DataFrame
+recharger_stats() None
+info_modele(tache, departement) dict
+predire(tache, departement, X) ndarray
+liste_evaluation(tache, departement) list[str]
+evaluation_modele(tache, departement, nom) DataFrame
+formulaire() dict
+champs(df, colonnes) dict
«script» AllegementImage
+TACHES: tuple
-repository: DataRepository
-registry: ModelRegistry
+snapshots_publies() tuple[str, str]
+colonnes_utiles(modeles) dict
+alleger(ecrire) None
«script» TrainingPipeline
+STEPS: tuple
+NB_SNAPSHOTS_GARDES: int
+TACHES: dict
-preprocessor: DataPreprocessor
-repository: DataRepository
-analyzer: StatAnalyzer
-registry: ModelRegistry
+run(steps, snapid, refresh) dict
+load_raw(snapid) dict
+audit(snapid, datasets) tuple
+preprocess(snapid, datasets) dict
+stats(snapid, processed) dict
+train(snapid, processed, version) dict
Données (src/data/)
DataRepository
+NB_DEPARTEMENTS: int
+SNAPSHOT_ID_FORMAT: str
-client: AdemeApiClient
-root: Path
-storage: Storage
-processed_root: Path
-processed_storage: Storage
-actif_path: Path
-departements: list
-stats_root: Path
-stats_storage: Storage
+refresh(size, nb_page) str
+departements() list[str]
+snapshot_actif() str|None
+modeles_actifs() str|None
+definir_snapshot_actif(snapshot_id, modeles_id) None
+remove_old_snapshots(a_garder) None
+list_snapshots() list[str]
+latest_snapshot() str
+load(snapshot_id) dict[str, DataFrame]
+save_processed(snapshot_id, data, nettoyage) None
+load_nettoyage(snapshot_id) dict
+load_processed(snapshot_id) dict[str, DataFrame]
+alleger_processed(snapshot_id, colonnes, ecrire) dict
+save_stats(snapshot_id, stats) None
+colonnes(snapshot_id) set[str]
+snapshot_precedent(snapshot_id) str|None
+save_audit(snapshot_id, audit) Path
+list_stats(snapshot_id) list[str]
+load_stat(snapshot_id, name) DataFrame
+list_stats_snapshots() list[str]
AdemeApiClient
+TIMEOUT: int
-API_URLS: dict
-API_NAMES: list
#params: dict
+get_api_url(name) str
+get_all_api_urls() dict
+get_full_datasets(code_departement, size, nb_page) dict
+get_dataset(name, code_departement, nb_page, **kwargs) DataFrame
+set_params(page, size, select, q, q_field, qs, params) None
+request_api(name, nb_page) DataFrame
3 (raw, processed, stats)
Stockage (src/storage/)
«abstract» Storage
+save(obj, path) None
+load(path) Any
+alleger(path, colonnes, ecrire) tuple[int, int]
LocalStorage
+root: Path
+save(obj, path) None
+load(path) DataFrame
+alleger(path, colonnes, ecrire) tuple[int, int]
Configuration (src/config/)
Settings
+tasks: dict
Non utilisée : aucun module ne l'importe et config/tasks.yaml est absent du dépôt
Traitement (src/processing/)
«module» analysis
+ANALYSES: dict (16 analyses)
+kpi_globaux(df)
+repartition_etiquettes(df)
... 14 autres analyses (df)
+qualite_donnees(data)
+apercu_donnees(data)
+bilan_nettoyage(nettoyage)
+colonnes_retirees(nettoyage)
StatAnalyzer
+noms() list[str]
+compute_all(data, nettoyage) dict[str, DataFrame]
«module» audit_colonnes
+colonnes_utilisees()
+decrire(serie, cible)
+auditer(df, colonnes_precedentes)
+afficher(snap, tout, manquantes, desc, ...)
«module» preprocessing
+nettoyer(df, bilan, colonnes_retirees)
+verifier_colonnes(df, colonnes, contexte)
+verifier_colonnes_obligatoires(df)
... 12 fonctions d'étape du nettoyage
DataPreprocessor
+appliquer_nettoyage: bool
+bilans: dict
+colonnes_retirees: dict
+rapport() dict
+fusionner(datasets) dict
+process(datasets) dict
lève
«exception (ValueError)» ColonnesManquantesError
+colonnes: list
Modèles (src/models/)
ModelRegistry
-root: Path
+save(tache, departement, snapid, model, metadata, evaluation) Path
+load(tache, departement, snapid) BaseModel
+load_meta(tache, departement, snapid) dict
+load_evaluation(tache, departement, nom, snapid) DataFrame
+list_snapshots(tache, departement) list[str]
+snapshots_complets(taches, departements) list[str]
+snapshot_source(version) str
+remove_old_snapshots(a_garder) None
+list_departements(tache, snapid) list[str]
sauvegarde / charge
«abstract» BaseModel
+model: Pipeline
+meilleurs_params: dict
+train(X, Y) None
+predict(X) ndarray
+evaluate(X, Y) dict
+tableaux_evaluation(X, y, y_train) dict[str, DataFrame]
+importance_variables(X, y, scoring, n) DataFrame
crée et entraîne (ClassificationModel, RegressionModel)
«module» features
+CIBLE_CLASSIFICATION: str
+CIBLE_REGRESSION: str
+PREFIXES_FUITE: tuple
+PREFIXES_CALCUL: tuple
+BASE_FORMULAIRE: list
+ISOLATION_PERCUE: list
+VARIABLES_FORMULAIRE: list
+preparer_xy(df, cible) tuple[DataFrame, Series]
ClassificationModel
+GRILLE_K: list
+LIBELLES: dict
+train(X, y) None
+predict(X) ndarray
+evaluate(X, y) dict
+tableaux_evaluation(X, y, y_train) dict
RegressionModel
+LIBELLES: dict
+BORNES_TRANCHES: list
+TAILLE_ECHANTILLON: int
+train(X, y) None
+predict(X) ndarray
+evaluate(X, y) dict
+tableaux_evaluation(X, y, y_train) dict
Légende
Héritage
Agrégation (possède)
Utilise
Dépend de
souligné : membre de classe
italique : méthode abstraite
«module» : fichier de fonctions, sans classe
«script» : point d'entrée en ligne de commande
+ public - privé # protégé




- Les modèles sont entraînés à part, puis chargés par l'application.
- Les données peuvent être mises à jour à tout moment par un nouvel appel à l'API.
- L'interface ne fait qu'afficher : les calculs sont réalisés en dehors.